

Proxmox Storage DRS -- Operator Manual

Installing, configuring and running it safely

Bernd Zeimetz bernd@bzed.de

2026-09-05

Rendered from docs/manual/*.md, SHA-256 0325bf278e1b67c06733be2863f04c87c0afd19cff13ad6f5dc3546f08ba4da0
(sha256sum --check docs/pve-storage-drs-manual.pdf.sha256 in the repository must succeed).

Copyright © 2026 Bernd Zeimetz. Licensed under the GNU Affero General Public License, version 3 or later.

Contents

What you need	5
Installing the package	5
Where the configuration lives	5
Where the configuration is <i>not</i>	5
Running the timer on exactly one host	6
First steps after installing	6

Configuration reference 6

schema_version	6
proxmox — the cluster API connection	6
proxmox.host	6
proxmox.port	6
proxmox.verify_ssl	6
proxmox.ca_file	6
proxmox.auth.username	7
proxmox.auth.password	7
proxmox.auth.token_id	7
proxmox.auth.token_secret	7
proxmox.ticket_refresh_seconds	7
proxmox.read_workers	7
prometheus — the metrics source	7
prometheus.url	7
prometheus.timeout_seconds	7
prometheus.username / prometheus.password	8
prometheus.bearer_token	8
metrics — Telegraf/InfluxDB name mapping	8
metrics.read_ops	8
metrics.write_ops	8
metrics.read_bytes	8
metrics.write_bytes	8
metrics.read_time_ns	8
metrics.write_time_ns	8
metrics.labels.vmid	8
metrics.labels.device	9
metrics.labels.node	9
metrics.rate_window	9
metrics.step	9
metrics.pvestatd_push_interval	9
window — the decision window	9
window.lookback	9
window.quantile	9
window.upper_quantile	9
window.min_coverage	10
load_weights — combining read/write and time/ops/bytes	10
load_weights.iotime	10
load_weights.ops	10
load_weights.bytes	10
load_weights.read_factor	10

load_weights.write_factor	10
groups — storage groups	10
groups[].name	10
groups[].storages[].id	11
groups[].storages[].capability_weight	11
groups[].storages[].reserve_factor	11
groups[].storages[].saturation_load	11
snapshot_reserve — the free-space floor	11
snapshot_reserve.factor	11
snapshot_reserve.min_free_bytes	11
snapshot_reserve.count_foreign_volumes	11
gates — deciding whether to act at all	12
gates.drift_threshold	12
gates.imbalance_threshold	12
gates.cooldown_per_disk	12
gates.cooldown_per_storage	12
migration — cost, bandwidth and the payback rule	12
migration.bwlimit_bytes_per_sec	12
migration.source_load_weight	12
migration.target_load_weight	12
migration.payback_horizon	12
migration.payback_ratio	13
migration.max_single_move_duration	13
migration.account_saferemove_wipe	13
migration.wipe_load_weight	13
migration.saturation_ceiling	13
migration.assume_thick_provisioning	13
objective — the solver’s trade-off weights	13
objective.spread_metric	13
objective.alpha_spread	13
objective.beta_move_count	14
objective.gamma_move_bytes_per_tib	14
objective.kappa_vm_affinity	14
objective.affinity_counts_pinned_disks	14
objective.reserve_violation_penalty	14
solver — which backend plans	14
solver.backend	14
solver.time_limit_seconds	14
solver.mip_gap	14
solver.heuristic_iterations	15
execution — how (and whether) moves actually happen	15
execution.mode	15
execution.max_concurrent_migrations	15
execution.max_migrations_per_run	15
execution.max_concurrent_per_storage	15
execution.max_replans_per_run	15
execution.abort_on_failure	15
execution.poll_interval_seconds	16
execution.locks.wait_timeout	16

execution.locks.poll_interval	16
execution.locks.on_timeout	16
execution.source_release.wait	16
execution.source_release.timeout	16
execution.time_windows[].days	16
execution.time_windows[].start / execution.time_windows[].end	16
exclude — what DRS never touches	16
exclude.vmsids	16
exclude.disks	17
exclude.storages	17
exclude.tags	17
exclude.skip_vms_with_snapshots	17
exclude.running_only	17
exclude.include_unused_disks	17
report	17
report.warn_pinned_load_fraction	17
state	17
state.path	17
forecast — history beyond the plain quantile	17
forecast.model	18
forecast.seasonal_lookback_days	18
forecast.holt_winters.seasonal_periods	18
forecast.holt_winters.trend	18
forecast.holt_winters.seasonal	18
forecast.holt_winters.residual_z	18
Verifying your setup	18
Running it	18
What each finding means	19
If it fails	19
Safety properties, exit codes, and what this build actually does	19
What is safe, unconditionally	19
Exit codes	20
What this build actually implements	20
Optional dependencies	20

What you need

- **Proxmox VE 9.2**, or a management host with network access to a PVE 9.x cluster's API (tcp/8006) — the tool does not need to run on a cluster member.
- **A Prometheus** already receiving PVE's per-disk blockstat metrics via the InfluxDB output plugin and Telegraf. If your cluster's dashboards already show per-VM disk I/O, this is already true; `pve-storage-drs verify-metrics` (page 2) confirms it before you rely on it. No new exporter or collector is required — see `IMPLEMENTATION_PLAN.md` section 3.1 if you want the detail of where the data comes from.
- **A PVE user or API token** with read access to VM and storage inventory and permission to call `move_disk`. An API token is preferred for unattended operation (`execution.mode: auto`); a user-name/password pair works for interactive use.

Installing the package

On a Debian trixie host (which is what Proxmox VE 9.x is built on):

```
apt install pve-storage-drs
```

This installs the `pve-storage-drs` executable, its manpage, the example configuration at `/usr/share/doc/pve-storage-drs/examples/drs.example.yaml`, and this manual and the specification as PDFs under `/usr/share/doc/pve-storage-drs/`.

`coinor-cbc` and `python3-pulp` are Recommends, not Depends: without them the tool still plans, using the dependency-free heuristic of `IMPLEMENTATION_PLAN.md` section 5.5, but a Debian install should have them — `apt install pve-storage-drs` pulls them in by default.

Where the configuration lives

`/etc/pve/drs.yaml`. On a cluster member this path is on `pmxcfs`, the cluster filesystem, so the file is automatically the same on every node — there is exactly one copy and no question of which one is authoritative. A management host that is not itself a cluster member simply keeps an ordinary file at that same path.

Two things follow from that placement, and both matter before you put a password in the file:

- **An edit is live on every node the instant it is saved.** There is no staging step. This is why configuration mistakes are always hard failures (see [30-safety-and-status.md](#)) rather than warnings, and why dry-run is the default execution mode.
- **Files under `/etc/pve` are group-readable by `www-data`** — the PVE web server's user. Check `ls -l /etc/pve/drs.yaml` on your cluster before deciding whether a plaintext password: is acceptable; the safer choice is an API token with `PVE_TOKEN_SECRET` set in the environment (or a `systemd` credential) rather than written into the file at all.

Copy `/usr/share/doc/pve-storage-drs/examples/drs.example.yaml` to `/etc/pve/drs.yaml` and edit it — every key is commented with its default and what it does. The full reference is [10-configuration.md](#).

Where the configuration is *not*

`state.path` (default `/var/lib/pve-storage-drs/state.json`) is deliberately **not** on `/etc/pve`. It is rewritten on every run, `pmxcfs` writes need cluster quorum, and the state it holds — cooldown timestamps, the load vector at the last balance, the run lock — is meaningful only to the one host that is actually running the timer. See [10-configuration.md#statepath](#).

Running the timer on exactly one host

Putting the configuration on `pmxcfs` does not make the tool cluster-aware. `state.json` is node-local: each node would keep its own cooldowns, its own drift baseline and its own run lock, which do not coordinate with each other. **Enable the systemd timer on exactly one host.** The one thing that *does* cross the cluster is a startup scan for in-flight `move_disk` UPIDs owned by the DRS user, which is what keeps two accidental instances from actively conflicting (they degrade to “slow and redundant” instead).

First steps after installing

1. `pve-storage-drs -c /etc/pve/drs.yaml verify-metrics` — confirms the configured metric and label names actually exist in your Prometheus. See [20-verifying-metrics.md](#).
2. Read [10-configuration.md](#) and adjust the storage groups, thresholds and weights for your cluster.
3. See [30-safety-and-status.md](#) for exactly which commands this build supports end to end today, and which still report “not implemented yet”.

Configuration reference

Every key `pve-storage-drs` reads is documented here: type, unit, default, what it interacts with, and what happens if you set it too high or too low. A key that exists in the schema but not here, or here but not in the schema, is a bug — `tests/unit/test_documentation.py` asserts the two never drift apart.

`config/drs.example.yaml` in the source tree (installed to `/usr/share/doc/pve-storage-drs/examples/drs.example.yaml`) carries the same defaults with inline comments; this page is the full explanation each comment points back to.

`schema_version`

Integer, required, no default.

The config format’s version. `pve-storage-drs` refuses to start on a `schema_version` whose **major** number it does not understand, naming the version it does understand in the error, rather than guessing at fields from a future incompatible format. Currently 1.

proxmox — the cluster API connection

`proxmox.host`

String, required, no default.

Any cluster member’s hostname or IP. The API is reached at `https://<host>:<port>/api2/json`.

`proxmox.port`

Integer, default 8006.

The PVE API port. Change this only if you have put something unusual (a reverse proxy) in front of the API.

`proxmox.verify_ssl`

Boolean, default `true`.

Whether to verify the API’s TLS certificate. Set `false` only for a self-signed certificate you cannot otherwise trust, and prefer `proxmox.ca_file` instead where possible — disabling verification also disables protection against a compromised network path to every node’s API.

`proxmox.ca_file`

String path or `null`, default `null`.

A CA bundle to verify the API’s certificate against, when it is not signed by a certificate already trusted by the host running `pve-storage-drs`.

proxmox.auth.username

String or null.

A PVE user, e.g. `drs@pve`. Required unless `proxmox.auth.token_id` is set. Username/password authentication is what the original requirement asked for; an API token (below) is preferred for `execution.mode: auto` because it needs no interactive ticket refresh and can be scoped to exactly the privileges section 3.5 of the plan lists.

proxmox.auth.password

String or null, default null, environment: `PVE_PASSWORD`.

Leave this null in the file and set `PVE_PASSWORD` in the environment (or a systemd credential) instead, especially when the config lives on `/etc/pve` and is therefore group-readable by `www-data`. A password given in the file is used as-is and is **not** overridden by the environment variable — see [00-installation.md](#) for why the file's own permissions matter here.

proxmox.auth.token_id

String or null, default null.

An API token id, e.g. `drs@pve!balancer`. Set this instead of username/password for unattended operation; a token needs no ticket refresh loop and can be revoked independently of the user's password.

proxmox.auth.token_secret

String or null, default null, environment: `PVE_TOKEN_SECRET`.

The secret half of the API token, with the same environment-variable preference as `proxmox.auth.password`.

proxmox.ticket_refresh_seconds

Duration, default 3000 (50 minutes).

How often a username/password ticket is refreshed. PVE tickets last approximately two hours; refreshing well before that avoids a mid-run authentication failure. Not used for API-token authentication, which needs no ticket.

proxmox.read_workers

Integer, default 12.

Size of the bounded thread pool used to fetch per-VM configuration — `GET /nodes/{node}/qemu/{vmid}/config` has no batch form, so this is what keeps a several-hundred-VM cluster's topology read from being serial. Raising it trades API server load for wall-clock time; 8–16 is the range the plan suggests. Too high a value on a small PVE API server can itself become the bottleneck.

prometheus — the metrics source**prometheus.url**

String, required, no default.

The base URL of your Prometheus, e.g. `http://prometheus.example.com:9090`.

prometheus.timeout_seconds

Duration, default 30.

Per-request timeout against Prometheus. Too low a value on a busy or long-range query makes `verify-metrics` and the load model fail spuriously; too high a value delays noticing that Prometheus is actually unreachable.

`prometheus.username / prometheus.password`

String or null, default null.

HTTP basic auth credentials, if your Prometheus requires them. Ignored when `prometheus.bearer_token` is set.

`prometheus.bearer_token`

String or null, default null.

A bearer token for Prometheus instances behind an auth proxy that expects one. Takes precedence over basic auth when both are set.

metrics — Telegraf/InfluxDB name mapping

Telegraf's naming is deployment-specific (IMPLEMENTATION_PLAN.md section 3.3), so every name below is configuration, never a constant. Run `pve-storage-drs verify-metrics` after changing any of them — see [20-verifying-metrics.md](#).

`metrics.read_ops`

String, default `blockstat_rd_operations`.

The Prometheus metric name for per-disk read operations/second (raw counter, before `rate()`).

`metrics.write_ops`

String, default `blockstat_wr_operations`.

As `metrics.read_ops`, for writes.

`metrics.read_bytes`

String, default `blockstat_rd_bytes`.

The Prometheus metric name for per-disk bytes read (raw counter).

`metrics.write_bytes`

String, default `blockstat_wr_bytes`.

As `metrics.read_bytes`, for writes.

`metrics.read_time_ns`

String, default `blockstat_rd_total_time_ns`.

The Prometheus metric name for cumulative read I/O time in nanoseconds. This is the **primary** load signal by default (`load_weights.iotime`) — see IMPLEMENTATION_PLAN.md section 4 for why I/O time, rather than IOPS or bytes, is the right default.

`metrics.write_time_ns`

String, default `blockstat_wr_total_time_ns`.

As `metrics.read_time_ns`, for writes.

`metrics.labels.vmid`

String, default `vmid`.

The Prometheus label carrying the VM id.

metrics.labels.device

String, default instance.

The Prometheus label carrying the drive id (`scsi0`, `virtio0`, ...). **Confirm this with `verify-metrics`.** PVE's own tag of this name collides with Prometheus's own scrape-target instance label, and many Telegraf configurations rename or overwrite it; `verify-metrics` warns loudly when this is still set to the literal string instance.

metrics.labels.node

String, default nodename.

The Prometheus label carrying the PVE node name.

metrics.rate_window

Duration, default 5m.

The inner range passed to `rate()`. Must be at least four times `metrics.pvestatd_push_interval`, or `rate()` sees too few points to be meaningful — `verify-metrics` measures the *observed* sample spacing of a live series and errors if this rule is violated against reality, not just against the declared `pvestatd_push_interval`.

metrics.step

Duration, default 5m.

The sampling step used for range queries (forecasting, coverage checks). Independent of `metrics.rate_window`, though the two are usually set equal.

metrics.pvestatd_push_interval

Duration, default 60s.

How often PVE's `pvestatd` pushes metrics — a PVE-side setting this tool cannot read from the API, so it must be declared here. `verify-metrics` cross-checks it against the *observed* sample spacing of a live series and errors if they disagree by more than 20%, which is what stops a stale declaration from silently invalidating the `rate_window` rule above.

window — the decision window

This is the period whose load is balanced. It is **not** the amount of history a forecaster needs to fit — see `forecast.model` below and `IMPLEMENTATION_PLAN.md` section 10.1.

window.lookback

Duration, default 24h.

The trailing window each disk's load is reduced over. Must be at least as long as the configured `forecast.model` needs (`forecast.model: holt_winters` at its default `seasonal_periods` needs 48h, which a 24h lookback can never supply — this is rejected at startup, not silently degraded).

window.quantile

Fraction in (0, 1), default 0.95.

The point-estimate quantile: each disk's load is the p95 of its raw signal over `window.lookback`, not the mean, so one traffic spike neither triggers nor suppresses a migration.

window.upper_quantile

Fraction in (0, 1), default 0.99.

The quantile the **optimizer** and the saturation guard actually consume — must be $\geq$ `window.quantile`. Being wrong in the direction of “busier than it looks” costs a slightly suboptimal balance; the other

direction risks migrating a disk onto a storage that is about to saturate, which is why the optimizer sees the more conservative bound rather than the point estimate.

`window.min_coverage`

Fraction in (0, 1], default 0.80.

A disk whose sample coverage over `window.lookback` falls below this fraction has its data rejected for this run; its last known load from `state.json` is used instead, flagged in the report. Never treated as zero — that would silently invite migrations *onto* a busy but under-sampled storage.

load_weights — combining read/write and time/ops/bytes

See `IMPLEMENTATION_PLAN.md` section 4 for the full blend-and-rescale formula; the summary is that `iotime/ops/bytes` are blended in normalized space and the whole result is rescaled back onto the in-flight-I/O scale, so $l_d = raw_t(d)$ exactly under the defaults below.

`load_weights.iotime`

Weight, default 1.0.

Weight on the I/O-time term ($rate(rd_total_time_ns + wr_total_time_ns) / 1e9$), the primary signal by default: it is Little's-law in-flight I/O, additive across disks and self-weighting between large sequential and small random access with no manual read/write tuning needed.

`load_weights.ops`

Weight, default 0.0.

Weight on the operations/second term. Non-zero only if you want IOPS to influence placement independently of I/O time.

`load_weights.bytes`

Weight, default 0.0.

Weight on the bytes/second term, for the same reason as `load_weights.ops`.

`load_weights.read_factor`

Weight, default 1.0.

Read/write asymmetry applied to the ops/bytes terms only (see `load_weights.write_factor`). Applying it to the `iotime` term as well double-counts, since I/O time already reflects the real read/write cost difference — do that only deliberately.

`load_weights.write_factor`

Weight, default 1.0.

As `load_weights.read_factor`, for the write side. A write-heavy RAID-6 array, for example, might set this above 1.0 on the ops/bytes terms to express the real cost asymmetry those terms cannot otherwise see.

groups — storage groups

`groups[].name`

String, required.

A group's name. A disk may only ever move between storages in its own group; there is no cross-group balancing (`IMPLEMENTATION_PLAN.md` section 5). Must be unique.

`groups[].storages[].id`

String, required.

A PVE storage id. A storage may belong to **at most one** group — belonging to two would make a disk's eligible destinations ambiguous, and config validation rejects it.

`groups[].storages[].capability_weight`

Weight, > 0 , default 1.0.

The storage's **relative** share of the group's load — $u_s = L_s / c_s$ is what actually gets equalized, so a storage with half the spindles of its peers can be given 0.5 and will correctly be assigned half the load. Only the *ratios* within a group matter; the absolute value is meaningless and it is not a capacity.

`groups[].storages[].reserve_factor`

Weight or null, default null (inherits `snapshot_reserve.factor`).

Per-storage override of the group-wide snapshot reserve factor, for a storage whose snapshot behaviour genuinely differs from its peers.

`groups[].storages[].saturation_load`

Positive number or null, default null.

The storage's approximate queue depth — the number of concurrent I/O requests it services before latency climbs super-linearly — in the same units as the load model (average in-flight I/O). Used only by the section 7.3 saturation guard on a migration's mirror. **Has no safe default:** an idle storage's observed load is not its capacity, so leaving this null (the default) simply disables that one advisory check for the storage; the hard bounds (`migration.max_single_move_duration`, the transient reserve invariant) always apply regardless. Obtain a real value from the array's documented queue depth, or by observing where latency actually starts climbing.

snapshot_reserve — the free-space floor

`snapshot_reserve.factor`

Weight ≥ 0 , default 2.0.

Keep this many times the largest disk on a storage free at all times, including *during* a migration, not merely before and after — PVE 9's volume-chain snapshots allocate a new full-size volume per snapshot, which is what this protects against. This is the constraint the tool never trades against balance (IMPLEMENTATION_PLAN.md section 5.3, (C5)).

`snapshot_reserve.min_free_bytes`

Size, default 0.

An absolute floor, applied as `reserve = max(factor * largest_disk, min_free_bytes)`. Matters when a storage's largest disk is small: with factor: 2.0 and a 10 GiB largest disk, the snapshot term alone would reserve only 20 GiB on a 20 TiB LUN.

`snapshot_reserve.count_foreign_volumes`

Boolean, default true.

Count volumes DRS does not manage (templates, ISOs, backups, other groups' disks, orphans) against a storage's used capacity. Strongly recommended: turning this off understates real usage and silently erodes the reserve it is meant to guarantee.

gates — deciding whether to act at all**gates.drift_threshold**

Fraction, default 0.10.

Re-planning is skipped until the load vector has moved by this much in L1, relative to the vector recorded at the last executed balance. Lower reacts sooner and migrates more; below roughly 0.05 the tool will start chasing noise on a busy cluster. Interacts with `gates.imbalance_threshold`: drift decides *whether to look*, imbalance decides *whether to act*.

gates.imbalance_threshold

Fraction, default 0.20.

The minimum relative spread across a group's storages before a plan is actually built. A group under this threshold is left alone even if it has drifted.

gates.cooldown_per_disk

Duration, default 24h.

A disk that moved within this long is pinned in place for planning purposes. Prevents a disk from being shuffled back and forth across two still-noisy storages.

gates.cooldown_per_storage

Duration, default 1h.

A storage involved in a migration within this long accepts no new incoming moves. **Must exceed the implied wipe time of that storage's largest disk** (`largest_disk / saferemove_throughput`) or the next run will plan onto a storage that is still draining and stall — `pve-storage-drs verify-storages` computes this and warns when the configured cooldown is too short.

migration — cost, bandwidth and the payback rule**migration.bwlimit_bytes_per_sec**

Size/s, default 209715200 (200 MiB/s).

Passed to `move_disk` as `bwlimit` (converted to KiB/s at the API call site — the API's own unit, never bytes/s). Also the divisor in the section 7.1 mirror-duration estimate used by the payback test.

migration.source_load_weight

Weight, default 1.0.

Additional in-flight I/O charged to the **source** storage while a mirror is running — a mirror is one sequential reader, so 1.0 is the natural value in the same units as the load model.

migration.target_load_weight

Weight, default 1.0.

As `migration.source_load_weight`, for the target (one sequential writer).

migration.payback_horizon

Duration, default 7d.

The horizon H over which a plan's imbalance reduction is assumed to persist — $\text{benefit} = \Delta E * H$. Setting this to 0 disables the payback test entirely (IMPLEMENTATION_PLAN.md section 11.1 rejects that at config-load time: `payback_horizon > 0` is required).

migration.payback_ratio

Weight > 0, default 10.0.

λ : a plan is only accepted if its total benefit is at least this many times its total migration cost. This is the numeric form of “migrating a very large disk might generate more traffic than it saves.”

migration.max_single_move_duration

Duration, default 6h.

A hard per-move ceiling on `duration_mirror` + `duration_wipe`. A move predicted to take longer than this is rejected outright regardless of the aggregate payback test.

migration.account_saferemove_wipe

Boolean, default true.

Include the post-move `saferemove` zeroing pass in cost and duration estimates. PVE’s LVM `saferemove` defaults to 10 MiB/s, so a 1.5 TiB disk’s cleanup (~44h) can dwarf its ~2.2h mirror; disabling this only if you have verified `saferemove` is off on every storage in a group understates real migration cost badly if that assumption is wrong.

migration.wipe_load_weight

Weight, default 1.0.

In-flight I/O charged to the source for the whole `saferemove` wipe duration — both in the cost model and in the section 7.3 saturation guard, where a draining move charges this to its source and nothing to its target. The zeroing pass is one sequential writer, so 1.0 is the natural value.

migration.saturation_ceiling

Fraction in (0, 1], default 0.85.

The fraction of a storage’s `saturation_load` (not of `capability_weight`) a move may drive it to. Inactive for any storage whose `saturation_load` is null — the default, since there is no safe way to infer it.

migration.assume_thick_provisioning

Boolean, default true.

Cost and reserve arithmetic use each disk’s *provisioned* size rather than its currently allocated size. Set false only for genuinely thin-provisioned storage, and note that allocation can *grow* during a move even then.

objective — the solver’s trade-off weights

See `IMPLEMENTATION_PLAN.md` section 5.4 for the full objective and section 14.3 for a worked demonstration of `beta_move_count` and `kappa_vm_affinity` choosing between competing plans.

objective.spread_metric

l1 or minmax, default l1.

How imbalance is measured: l1 (sum of each storage’s deviation from the group’s target utilization) or minmax (only the single hottest storage). l1 is the default because minmax is indifferent to a second nearly-as-bad storage once the worst one is fixed.

objective.alpha_spread

Weight, default 1.0.

Weight on the imbalance term. This and the other three weights below share one scale — see the worked arithmetic in `IMPLEMENTATION_PLAN.md` section 14.3 for what “a 0.25-request reduction” actually costs against one move.

objective.beta_move_count

Weight, default 0.25.

Penalty per migration — the tunable implementation of “minimize the number of migrations” (IMPLEMENTATION_PLAN.md’s “Requirement interpretation” note: a strict minimum would refuse a second cheap move that halves the remaining imbalance, which is not desired). Raising it trades balance quality for fewer, larger moves.

objective.gamma_move_bytes_per_tib

Weight per TiB, default 0.05.

Penalty per TiB actually migrated, biasing against moving large disks specifically (as opposed to beta_move_count, which only counts moves).

objective.kappa_vm_affinity

Weight, default 0.50.

Penalty per extra storage a VM’s disks are spread across, counted only **within** a group (a VM split across two groups is structural and cannot be repaired by any migration, so it is not counted). A soft preference: a strong imbalance or a capacity constraint can legitimately override it.

objective.affinity_counts_pinned_disks

Boolean, default false.

Whether a disk that cannot move this run (snapshot-blocked, excluded, or on a locked VM) still counts toward kappa_vm_affinity. Default false so one unreachable disk cannot veto good placement of the rest of its VM’s disks; note efidisk0/tpmstate0 are *not* in this pinned set on PVE 9.2 — they move online.

objective.reserve_violation_penalty

Weight, default 1000.0.

A **floor**, not the value actually used, for the single-stage big-M fallback solve path (IMPLEMENTATION_PLAN.md section 5.3, option 2): the engine computes a provably-dominant P from the group’s own load and disk sizes at solve time and uses `max(configured, computed)`, warning when it had to raise it. The default lexicographic two-stage solve (option 1, and the default) needs no penalty at all and is unaffected by this key.

solver — which backend plans**solver.backend**

One of auto, cpsat, cbc, heuristic; default auto.

auto prefers CP-SAT, falls back to CBC (the packaged solver path via python3-pulp + coinor-cbc), then the dependency-free heuristic. Force a specific backend only to reproduce or compare a result — heuristic is also the automatic choice for a group too large for either MILP backend to solve within solver.time_limit_seconds.

solver.time_limit_seconds

Duration, default 60.

Wall-clock budget for a MILP solve attempt before falling back to the heuristic. Never emits a partial/unvalidated assignment on timeout — it falls back cleanly instead.

solver.mip_gap

Fraction, default 0.02.

Acceptable optimality gap for the MILP solve. `IMPLEMENTATION_PLAN.md` section 5.5's coefficient-scaling error bound is three orders of magnitude below this, so it cannot itself change which plan is selected.

`solver.heuristic_iterations`

Integer > 0, default 5000.

Iteration budget for the heuristic's local-search descent phase (section 5.5). Higher can find a better local optimum on a large group at the cost of run time.

execution — how (and whether) moves actually happen**`execution.mode`**

One of dry-run, confirm, auto; default dry-run.

dry-run (the default) prints the plan and changes nothing. `confirm` executes one move at a time with a prompt before each. `auto` executes unattended, subject to the concurrency and time-window settings below. `--mode` on the command line overrides this for one run; moving *toward* less safety is logged at warning level (`IMPLEMENTATION_PLAN.md` section 11.3).

`execution.max_concurrent_migrations`

Integer >= 1, default 1.

How many moves may be in flight across the whole run at once. Above 1 requires the *generalized* transient reserve invariant (section 8.1): several disks can land on one storage at once, and none of their sources release space until each individually completes.

`execution.max_migrations_per_run`

Integer >= 1, default 5.

A ceiling on how many moves one invocation executes, independent of how many the plan contains — the remainder waits for the next run.

`execution.max_concurrent_per_storage`

Integer >= 1, default 1.

Caps concurrent moves touching one storage, counting it as either source or target. At the default of 1, two moves targeting the same storage serialize automatically and the generalized transient invariant collapses to its simple single-move form. Raising it should be a deliberate act on a storage with real spare headroom — two moves off the same source also means two concurrent `saferemove` wipes sharing one throttle.

`execution.max_replans_per_run`

Integer >= 0, default 3.

A cap on how many times one run may abandon its current plan and re-plan from newly observed state (section 9.2) — a mismatch between the plan and reality (a VM live-migrated mid-plan, a lock appeared) is normal, but a cluster churning faster than the engine can plan is a condition for a human, not for indefinite retrying.

`execution.abort_on_failure`

Boolean, default `true`.

On a failed move, stop the run and report rather than continuing with the remaining plan. The cluster is left in a valid intermediate state either way; the next run re-plans from reality.

execution.poll_interval_seconds

Duration, default 10.

How often an in-flight move_disk task's status is polled.

execution.locks.wait_timeout

Duration, default 4h.

How long to wait for a VM's config lock (any non-empty value — the set is treated as open-ended and never whitelisted) to clear before applying execution.locks.on_timeout. Generous by default: a backup of a large VM can easily run for hours, and that is a normal, not exceptional, condition.

execution.locks.poll_interval

Duration, default 30s.

How often the lock is re-checked while waiting.

execution.locks.on_timeout

One of skip, abort; default skip.

What happens when execution.locks.wait_timeout elapses: skip this move and continue with the rest of the plan, or abort the run. Never “force” — that option does not exist, deliberately.

execution.source_release.wait

Boolean, default true.

Wait for the source volume of a completed move to actually disappear (and the VM to be unlocked) before considering the move done, rather than trusting the move_disk task's own OK status — with safermove enabled, OK does not mean the source space is back yet. Set false only on storages verified not to wipe.

execution.source_release.timeout

Duration, default 48h.

Bound on the wait above, sized for a multi-TiB disk wiping at the default 10 MiB/s. Must be at least largest_disk / safermove_throughput for every storage where safermove is on — verify-storages checks this.

execution.time_windows[].days

List of mon..sun, default: every day.

Which days a time window applies on. Applies only to execution.mode: auto — dry-run and confirm are not time-restricted, since neither changes anything without a human already present.

execution.time_windows[].start / execution.time_windows[].end

HH:MM strings, required together, must differ.

The window auto mode is permitted to execute moves in. A window may cross midnight (start: "22:00", end: "06:00"); start == end is rejected as ambiguous — it would silently mean either “never” or “always”.

exclude — what DRS never touches**exclude.vmid**

List of integers, default [].

VM ids whose disks are never moved.

exclude.disks

List of "vmid:device" strings, default [], e.g. ["101:scsi1"].

Individual disks excluded regardless of their VM.

exclude.storages

List of storage ids, default [].

Storages excluded as both source and target — effectively removes them from their group without editing groups.

exclude.tags

List of PVE tags, default ["no-drs"].

A VM carrying any of these tags opts out entirely.

exclude.skip_vms_with_snapshots

Boolean, default true.

Pre-filter disks with a snapshot or an unreferenced companion volume before they reach the API. Setting this false does **not** make such a move work — PVE rejects `move_disk delete=1` on a snapshotted volume regardless; it only removes the early, informative warning. Keep this true.

exclude.running_only

Boolean, default true.

Only running VMs generate load to balance; a stopped VM's disks are moved only if a capacity constraint requires it. Setting this false widens the movable set to stopped VMs' disks for capacity purposes but they still contribute zero load.

exclude.include_unused_disks

Boolean, default true.

Whether detached (unusedN) volumes are movable. They carry zero load, so the solver only relocates one to repair a capacity violation — never for balance, since there is no benefit to buy. Set false to pin them in place entirely; they still count via `snapshot_reserve.count_foreign_volumes` either way.

report**report.warn_pinned_load_fraction**

Fraction in (0, 1], default 0.25.

If pinned load (snapshots, locks, exclusions) exceeds this fraction of a group's total, the residual imbalance may be structural rather than a planning failure, and the report says so alongside the best achievable spread given the pins.

state**state.path**

File path, default `/var/lib/pve-storage-drs/state.json`.

Local disk, one copy per host, deliberately never `/etc/pve` — see [00-installation.md](#). Losing this file is safe but not free: cooldowns and the drift baseline reset, so the next run may migrate sooner than intended. Treat it as state to back up, not as a cache.

forecast — history beyond the plain quantile

See `IMPLEMENTATION_PLAN.md` section 10 and `proxmox_storage_drs/forecast.py`.

forecast.model

One of quantile, seasonal_naive, holt_winters; default quantile.

quantile needs only window.lookback and no model fitting. seasonal_naive and holt_winters need more history than the decision window alone — see below — and pve-storage-drs refuses to start if window.lookback (or your Prometheus retention) cannot supply it, rather than silently falling back.

forecast.seasonal_lookback_days

Days, default 7.

History seasonal_naive needs: same-hour-of-day samples across this many days. 0 disables seasonal_naive (falls back to quantile, logged).

forecast.holt_winters.seasonal_periods

Integer > 0, default 288 (24h at a 5-minute step).

Samples per seasonal cycle. Combined with metrics.step, this determines holt_winters's required history: $2 * \text{seasonal_periods} * \text{metrics.step}$ — 48h at the defaults, which a 24h window.lookback can never satisfy.

forecast.holt_winters.trend

One of add, mul, none; default add.

The trend component passed to the underlying Holt-Winters fit (statsmodels, an optional dependency — see [30-safety-and-status.md](#)).

forecast.holt_winters.seasonal

One of add, mul, none; default add.

The seasonal component passed to the same fit.

forecast.holt_winters.residual_z

Weight, default 2.0.

The upper bound is $\text{point_estimate} + \text{residual_z} * \text{stdev}(\text{residuals})$ from the in-sample fit — this is what the optimizer actually consumes (see window.upper_quantile for the equivalent under the quantile model).

Verifying your setup

Run this before relying on any plan. Telegraf and InfluxDB naming varies by deployment, so pve-storage-drs never assumes the defaults in [10-configuration.md](#) are correct for your cluster — verify-metrics is what checks them against the live Prometheus instead.

Running it

```
pve-storage-drs -c /etc/pve/drs.yaml verify-metrics
```

A successful run looks like this (labels and values are illustrative):

```
[ info] metrics.read_ops = 'blockstat_rd_operations' exists
[ info] metrics.write_ops = 'blockstat_wr_operations' exists
[ info] metrics.read_bytes = 'blockstat_rd_bytes' exists
[ info] metrics.write_bytes = 'blockstat_wr_bytes' exists
[ info] metrics.read_time_ns = 'blockstat_rd_total_time_ns' exists
[ info] metrics.write_time_ns = 'blockstat_wr_total_time_ns' exists
[ info] blockstat_rd_operations: sample series labels {'vmid': '101', 'instance': 'scsi0', 'nodename': 'pve01'}
```

...

OK: verify-metrics found no blocking problems

Exit status is 0 when every check passes, 1 when any check reports an error-level finding (see below).
 --json emits the same information as one JSON object instead — see [30-safety-and-status.md](#) for exit codes in general.

What each finding means

verify-metrics runs six checks, in this order (IMPLEMENTATION_PLAN.md section 3.3):

1. **Metric names exist.** Each of `metrics.read_ops`, `metrics.write_ops`, `metrics.read_bytes`, `metrics.write_bytes`, `metrics.read_time_ns` and `metrics.write_time_ns` is looked up against Prometheus's own `/api/v1/label/__name__/values`. A missing name is an **error** — fix the name in `10-configuration.md`'s `metrics.*` section to match what your Telegraf actually emits.
2. **Sample series and labels.** One instant query per metric, printing its full label set so you can see at a glance whether `metrics.labels.vmid`, `metrics.labels.device` and `metrics.labels.node` are really present and non-empty on a live series. A metric that exists but currently has no series is a **warning**, not an error — it may simply mean nothing has generated that kind of I/O recently.
3. **Configured labels present.** If `metrics.labels.vmid`/`metrics.labels.device`/`metrics.labels.node` do not appear (or are empty) on the sample series, that is an **error**: the load model has nothing to join disks on.
4. **The instance collision.** If `metrics.labels.device` is still the literal string `instance`, a **warning** — PVE's own per-disk tag has this name and it collides with Prometheus's unrelated scrape-target instance label. Many Telegraf configurations rename or overwrite one of the two; confirm which one survived before trusting this.
5. **Per-disk coverage.** For every (`vmid`, `device`) pair seen on `metrics.read_ops`, the fraction of expected samples actually present over `window.lookback` is measured. Below `window.min_coverage`, a **warning** naming the disk — that disk's own data will be rejected by the load model at plan time and its last known load from `state.json` used instead.
6. **Observed sample spacing.** The modal gap between consecutive samples of a live series is measured directly, rather than trusted from `metrics.pvestatd_push_interval`. Disagreeing by more than 20%, or `metrics.rate_window` being below four times what was actually observed, is an **error** — this is what stops a stale `pvestatd_push_interval` declaration from silently breaking the `rate()` window it is supposed to protect.

If it fails

- **A metric does not exist:** check your Telegraf `influxdb_listener` (or equivalent) configuration for the actual measurement/field names it is producing, and update `metrics.*` to match.
- **A label is missing or empty:** the sample series printed by check 2 is the fastest way to see what labels Prometheus actually has for that series — update `metrics.labels.*` to name the real ones.
- **Coverage is low for specific disks:** check that Telegraf is actually scraping every node, and that `window.lookback` is not longer than your Prometheus retention.
- **Spacing disagrees:** measure `pvestatd`'s real push interval on a node (its `systemd` timer, or the interval visible in `/etc/pve/status.cfg` if set there) and correct `metrics.pvestatd_push_interval`.

Safety properties, exit codes, and what this build actually does

What is safe, unconditionally

These hold regardless of `execution.mode`, and are not configurable away (`.agents/domain-invariants.md`):

- **Dry-run is the default.** A missing or unparseable `execution.mode` means dry-run, never automatic execution.
- **The snapshot reserve is never traded for balance.** `pve-storage-drs` will leave a group imbalanced rather than breach `snapshot_reserve.factor/min_free_bytes` on any storage, and that reserve holds *during* a migration, not merely before and after it.
- **Nothing is ever deleted automatically.** An orphaned volume left by a failed move is reported, never removed. The operator deletes.
- **An explicitly named configuration that cannot be read or fails validation is a hard failure,** never a silent fallback to `/etc/pve/drs.yaml`.

Exit codes

Code	Meaning
0	Success. A run that stopped at a gate, found nothing worth moving, or (for <code>verify-metrics</code>) found no error-level finding.
1	The run failed: invalid configuration, an unreachable Prometheus or PVE API, a failed check, or a command not yet implemented in this build (see below).
2	Usage error on the command line.

What this build actually implements

`IMPLEMENTATION_PLAN.md` section 12 lays out nine phases. This is a work in progress; being honest about exactly where it stands matters more than a document that reads as if the tool were finished:

Command	Status
<code>--version</code> , <code>--help</code> , <code>--manual</code>	Implemented.
<code>verify-metrics</code>	Implemented: all six <code>IMPLEMENTATION_PLAN.md</code> section 3.3 checks, human and <code>--json</code> output.
<code>plan</code>	Not implemented yet — needs <code>pve.py/topology.py</code> (phase 2) and the load model, gates, scheduler and solver (phases 3-6).
<code>show-load</code>	Not implemented yet — needs <code>pve.py/topology.py</code> (phase 2).
<code>explain</code>	Not implemented yet — needs the same, plus the payback and scheduling machinery it explains.
<code>verify-storages</code>	Not implemented yet — needs <code>pve.py</code> (phase 2).
<code>apply</code>	Not implemented yet — needs <code>execute.py</code> (phase 7 onward).

Configuration loading and validation (this whole manual’s [10-configuration.md](#)) is complete and exercised by every command, including the ones above that are not yet implemented: a config error is reported and the run exits 1 before reaching the “not implemented” message, so validating a configuration file does not require waiting for the rest of the engine.

Running a command that is not implemented yet prints a message naming the plan section it belongs to, and exits 1:

```
$ pve-storage-drs plan
pve-storage-drs: 'plan' is not implemented yet in this development build; see IMPLEMENTATION_PLAN.md section 12 for the phase it belongs to
```

Optional dependencies

`pve-storage-drs` runs, plans and (once implemented) executes with only `requests`, `ruamel.yaml` and `jsonschema` installed. Two dependencies are optional and are imported only where they are used, never at module level:

- **pulp + coinor-cbc** (Debian Recommends) — the packaged MILP solver path. Without it, the solver falls back to the dependency-free heuristic.
- **statsmodels** (Debian Suggests) — needed only for `forecast.model: holt_winters`. Without it, that model logs a warning and falls back to quantile automatically rather than failing.